

PERTEMUAN 1

POINTER

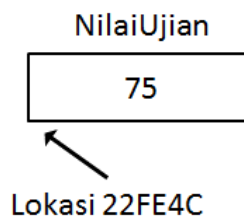
A. Tujuan Pembelajaran

Mahasiswa mengerti definisi pointer dan dapat menggunakan dalam program sederhana.

B. Uraian Materi

Pengertian Pointer

Tiap variabel yang didefinisikan di program akan diberikan satu tempat di memori. Nilai variabel disimpan di lokasi tersebut.



Gambar 1.1 Variabel *NilaiUjian* disimpan di salah satu lokasi di memori

Untuk melihat di lokasi mana nilai ini disimpan, dapat dipakai operator & atau *reference*, yaitu mendapatkan alamat dari variabel tersebut.

```
int NilaiUjian = 75;
```

```
cout << &NilaiUjian;
```

Keluaran dari potongan program di atas adalah alamat dari memori tempat sistem operasi menyimpan variabel *NilaiUjian*. Contoh adalah semacam berikut.

```
0x22fe4c
```



Catatan:

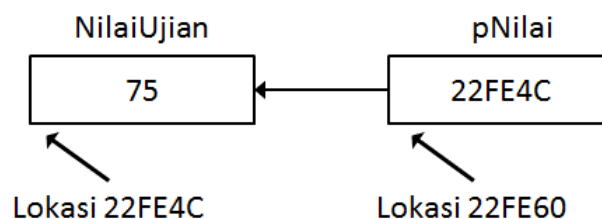
Keluaran program anda dapat berbeda dengan contoh di atas karena alamat memori tergantung perangkat keras, sistem operasi, software yang sedang berjalan, dan banyak faktor lain.

Kita dapat menyimpan alamat ini dalam suatu variabel lain. Variabel tempat kita menyimpan alamat variabel lain disebut pointer. Dengan kata lain, pointer adalah suatu variable yang berisi alamat memori dari suatu variable lain.

Contoh Program 1

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     int NilaiUjian = 75;
5     int *pNilai;
6     pNilai=&NilaiUjian;
7     cout << &NilaiUjian<<endl;
8     cout << &pNilai<<endl;
9     cout << pNilai<<endl;
10    cout << *pNilai<<endl;
11 }
```

Program di atas dapat digambarkan sebagai berikut.



Gambar 1.2 Variabel NilaiUjian dan pointer pNilai

Variabel pNilai merupakan variabel pointer. Penulisannya dilakukan dengan menambahkan tanda bintang (*) di depan nama variabel. Variabel pNilai diisi dengan alamat dari variabel NilaiUjian dengan perintah pNilai = &NilaiUjian. Sedang variabel NilaiUjian berisi angka 75.

Keluaran dari program di atas adalah seperti di bawah.

0x22fe4c

0x22fe60

0x22fe4c

75

Angka pertama dan kedua adalah alamat dari NilaiUjian dan alamat dari pNilai, berturut-turut. Angka ketiga adalah nilai yang disimpan di variabel pNilai yaitu alamat NilaiUjian, sehingga tampilan angka pertama dan ketiga nilainya sama. Angka terakhir adalah nilai dari NilaiUjian, yakni 75.

Deklarasi Variabel Pointer

Bentuk umum deklarasi variabel pointer adalah sebagai berikut.

tipe_data* namaVariabel

Contoh :

int* pNilai;

Ada tiga cara penulisan deklarasi pointer. Perbedaan ada pada penulisan spasi.

int* pNilai;

int *pNilai;

int * pNilai;

Ketiga cara penulisan di atas benar menurut C++.

Agar pointer menunjuk ke suatu variabel maka perlu diisi dengan alamat dari variabel yang bersangkutan. Perlu diperhatikan bahwa variabel dan pointernya harus mempunyai tipe data yang sama.

Contoh :

```
int Usia;  
  
int* pUsia;  
  
pUsia = &Usia;
```

Tanda * menunjukkan bahwa variabel dideklarasikan sebagai pointer. Variabel pointer pUsia mempunyai tipe data integer dan menunjuk ke sebuah obyek (Usia) yang juga memiliki tipe integer.

Tanda & menandakan operator alamat, merupakan operator *unary* yang mengembalikan alamat dari operand.



Catatan:

Operator *unary* adalah operator yang hanya mempunyai satu operand. Contoh adalah operator – (minus) seperti pada $-A$. Bila A bernilai positif, maka $-A$ adalah negatif. Sebaliknya, jika A negatif maka nilai $-A$ adalah positif.

Operator unary & memberikan alamat variabel operand. Perintah &A akan memberikan alamat dari variabel A (operand) di dalam memori. Variabel A harus sudah dideklarasikan terlebih dahulu. Operator & sering disebut sebagai *reference*.

Sebaliknya, operator * memberikan nilai dari alamat yang ditunjuk oleh operand di mana operand di sini adalah variabel pointer. Perintah *pA memberikan nilai dari alamat yang disimpan di pointer pA. Operator * sering disebut sebagai *dereference*.

Contoh Program 2

1	#include <iostream>
2	using namespace std;
3	int main(void)
4	{

5	int A=25; // deklarasi integer
6	int* pA; // deklarasi pointer ke integer
7	pA=&A; // pointer ke alamat A
8	cout<<"A = "<<A<<endl; // tampilkan nilai A
9	cout<<"&A = "<<&A<<endl; // tampilkan alamat A
10	cout<<"pA = "<<pA<<endl; //tampilkan nilai pA
11	cout<<"&pA = "<<&pA<<endl;//tampilkan alamat pA
12	cout<<"*pA = "<<*pA<<endl;//nilai yang ditunjuk pA
	}

Hasil tampilan adalah sebagai berikut.

A = 25

&A = 0x22fe3c

pA = 0x22fe3c

&pA = 0x22fe30

*pA = 25

Di baris ke 7 dalam program di atas, variabel pointer pA diisi alamat A dengan perintah pA = &A. Sehingga pA jika ditampilkan akan berisi alamat A. Sedang *pA adalah nilai yang ditunjuk oleh pointer pA, yaitu nilai A.



Peringatan:

Perhatikan bahwa tampilan pA, &pA, maupun *pA menampilkan nilai yang berbeda-beda. Contoh di atas menggambarkan hal ini dengan jelas. Ini harus dimengerti benar agar pemakaian pointer dapat tepat dilakukan.

Contoh Program 3

1	#include <iostream>
2	using namespace std;
3	int main(void)
4	{
5	int A=25; // deklarasi integer
6	int* pA; // deklarasi pointer ke integer
7	pA=&A; // mengarahkan pointer ke alamat A
8	cout<<"A="<<A<<endl; // tampilkan nilai A
9	cout<<"*pA="<<*pA<<endl; //nilai yang ditunjuk pA
10	A=35; // nilai A diubah menjadi 35
11	cout<<"A="<<A<<endl; // tampilkan nilai A
12	cout<<"*pA="<<*pA<<endl; //nilai yang ditunjuk pA
13	*pA+=10; // nilai A ditambah 10
14	cout<<"A="<<A<<endl; // tampilkan nilai A
15	cout<<"*pA="<<*pA<<endl; //nilai yang ditunjuk pA
16	}

Hasil keluaran dari program di atas adalah:

A=25

*pA=25

A=35

*pA=35

A=45

*pA=45

Di baris 7, pointer pA diisi dengan alamat dari variabel A. Maka A dan *pA ketika ditampilkan akan bernilai sama yaitu 25.

Di baris 10, variabel A diubah nilainya menjadi 35. Maka nilai *pA juga akan berubah nilainya menjadi 35. Lihat tampilan untuk baris 11 (nilai A) dan baris 12 (nilai *pA).

Di baris 13 nilai *pA ditambah 10 sehingga menjadi 45. Maka nilai A juga berubah menjadi 45. Perhatikan hasil tampilan untuk baris 14 (nilai A) dan baris 15 (nilai *pA).

Dari program di atas dapat dilihat bahwa mengubah nilai A dapat dilakukan lewat variabel A maupun lewat variabel pointer pA. Ini karena pA menunjuk ke alamat memori yang sama dengan A.

Contoh Program 4

1	#include <iostream>
2	using namespace std;
3	int main(void)
4	{
5	string makanan = "Bakso"; // Tipe string
6	string* ptr = &makanan; // variabel pointer
7	cout << makanan << endl; // tampilkan nilai
8	cout << *ptr << endl; // tampilkan nilai
9	cout << &makanan << endl; // tampilkan alamat
10	cout << ptr << endl; // tampilkan alamat
11	}

Tampilan dari program di atas adalah sebagai berikut.

```
Bakso
Bakso
0x22fe20
0x22fe20
```

Program membuat variabel pointer dengan nama ptr, yang menunjuk ke variabel bertipe string. Perhatikan bahwa tipe data variabel pointer harus sama dengan tipe data variabel yang akan ditunjuk.

Program memakai operator & untuk mengambil alamat dari variabel makanan dan menyimpannya di variabel pointer ptr. Selanjutnya, ptr sudah memiliki alamat variabel makanan. Selanjutnya dapat ditebak bahwa tampilan makanan dan *ptr akan sama yaitu nilai yang tersimpan di variabel makanan. Dan tampilan &makanan dan ptr akan sama yaitu alamat dari variabel makanan.

Latihan

Soal 1

```
int panjang=25, lebar=11, A=10;
int* pA;
pA = &panjang;
cout<<"Nilai: "<<*pA<<endl;
pA = &lebar;
cout<<"Nilai: "<<*pA<<endl;
pA = &A;
cout<<"Nilai: "<<*pA<<endl;
A = panjang * lebar;
cout<<"Nilai: "<<*pA<<endl;
```

Apa tampilan yang dihasilkan oleh potongan program di atas? Jelaskan mengapa 4 perintah yang sama `cout<<"Nilai: "<<*pA<<endl;` dapat menghasilkan tampilan yang berbeda-beda.

Soal 2

```
double panjang=25;
```

```
int *pA;
```

```
pA = &panjang;
```

Potongan program di atas menghasilkan kesalahan saat kompilasi di baris ke 3. Mengapa?

Soal 3

```
int panjang=25;
```

```
int* pA, pB;
```

```
pA = &panjang;
```

```
pB = &panjang;
```

Potongan program di atas menghasilkan kesalahan saat kompilasi di baris ke 4. Jelaskan.

Pointer dan Larik 1 Dimensi

Pointer dapat digunakan untuk menunjuk ke larik (*array*) dan selanjutnya pointer dapat dipakai untuk mengakses elemen-elemen larik. Contoh, dideklarasikan variabel larik integer dengan 3 anggota dan pointer sebagai berikut.

```
int A[]={14,16,19};
```

```
int *pA;
```

Pemberian nilai (*assignment*) pointer ke larik 1 dimensi dapat dilakukan dengan dua cara.

```
pA = A;
```

atau

```
pA = &A[0];
```

Untuk mengarahkan pointer ke anggota pertama dari suatu larik, dapat dilakukan dengan memberikan variabel larik tanpa indeks, sebagaimana contoh di atas, yaitu perintah `pA=A`. Dapat juga dilakukan dengan memberi alamat anggota pertama dari larik, yaitu anggota dengan indeks 0. Contoh adalah `pA=&A[0]`. Kedua perintah tersebut diperlakukan sama oleh C++.



Catatan:

Nama larik adalah constant pointer ke elemen pertama dari larik. Karenanya jika dideklarasikan `int A[10]`, maka `A` adalah sama dengan `&A[0]`. Yaitu alamat dari anggota pertama dari larik. Misal:

```
int A[10]={1,2,3,4,5,6,7,8,9,10};
cout<<A<<endl;
```

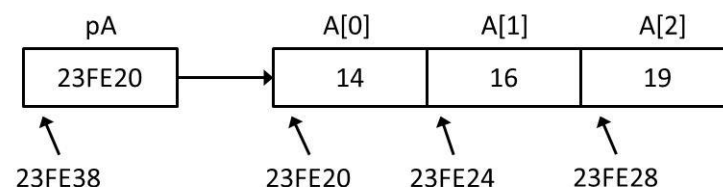
Maka akan ditampilkan alamat elemen pertama `A`, yaitu

`0x22fe30`

(Alamat bisa berbeda, tergantung komputer)

Perhatikan yang ditampilkan bukan nilai `A[0]`, tapi alamat dari `A[0]`.

Penunjukan alamat oleh variabel pointer dapat digambarkan sebagai berikut.



Gambar 1.3 Pointer ke anggota pertama dari larik

Alamat dari `A[0]` adalah `0x23fe20`. Dengan perintah `pA=A` atau `pA=&A[0]` maka `pA` akan berisi alamat dari `A[0]`.

Catatan:

Membuat larik tidak harus menyebutkan jumlah anggota. C++ akan menghitung jumlah anggota yang dituliskan, dan membuat larik sesuai dengan jumlah anggota yang dirinci di perintah tersebut. Contoh perintah `int A[]={14,16,19}` akan membuat larik dengan anggota 3 buah integer karena ada tiga anggota yang dituliskan yaitu 14, 16, dan 19.

Contoh Program 1

```
1  #include <iostream>
2  using namespace std;
3  int main(void) {
4      int *pTgl;    // deklarasi pointer
5      int tglLahir[]={19,4,1980}; // deklarasi larik
6      pTgl=tglLahir;    // pemberian nilai
7      //Menampilkan isi array dengan pointer
8      for(int i=1; i<=3; i++) {    // looping
9          cout<<*pTgl<<endl; //tampilkan nilai
10         pTgl++;    // increment pointer
11     }
12     cout << endl;    // satu baris kosong
13     pTgl=&tglLahir[0]; // pemberian nilai
14     cout<<*pTgl<<endl;    // tampilkan nilai
15     cout<<*(pTgl+1)<<endl;    // alamat tambah 1
16     cout<<*(pTgl+2)<<endl;    // alamat tambah 2
    }
```

Tampilan program di atas:

```
19
4
1980

19
4
1980
```

Di baris ke 6 pTgl diberi nilai alamat tglLahir dengan perintah pTgl=tglLahir. Ini identik dengan memberi alamat tglLahir[0]. Dalam pengulangan ditampilkan nilai yang ditunjuk pointer pTgl. Pertama kali ini akan menampilkan tglLahir[0] yaitu 19. Perintah increment di baris 10 akan membuat pTgl berisi alamat tglLahir[1] sehingga perintah cout berikutnya akan menampilkan angka 4. Increment sekali lagi membuat pTgl menunjuk ke alamat tglLahir[2]. Perintah cout berikutnya menampilkan angka 1980.

Baris 13 mengembalikan pointer ke alamat tglLahir[0]. Perintah *(pTgl+1) akan menampilkan nilai larik tglLahir di indeks 1. Perintah *(pTgl+2) akan menampilkan nilai larik indeks 2.

Contoh Program 2

1	#include<iostream>
2	using namespace std;
3	int main(void){
4	int angka[5];
5	int * p;
6	p = angka; *p = 10;
7	p++; *p = 20;

8	<code>p = &angka[2]; *p = 30;</code>
9	<code>p = angka + 3; *p = 40;</code>
10	<code>p = angka; *(p+4) = 50;</code>
11	<code>for (int n=0; n<5; n++)</code>
12	<code>cout << angka[n] << " ";</code>
13	<code>}</code>

Program di atas membuat deklarasi angka sebagai larik integer (bilangan bulat) dengan 5 anggota. Kemudian diperlihatkan beberapa macam cara untuk mengakses larik.

Keluaran dari program di atas adalah seperti di bawah.

10 20 30 40 50

Terlihat nilai dari larik angka dapat berubah dengan cara mengakses nilainya dengan memakai pointer.

C. Latihan

Apa tampilan potongan program berikut? (Buatlah program lengkap untuk dapat menjalankan program-program berikut)

Soal 1.

```
int *P;

int A[5]={2,1,7,0,5};

P=A;

cout<<*P<<endl;

P++;

cout<<*P<<endl;
```

Soal 2.

```
int *P;  
int A[5]={2,1,7,0,5};  
P=A;  
cout<<*P++<<endl;  
cout<<*P<<endl;
```

Soal 3.

```
int *P;  
int A[5]={2,1,7,0,5};  
P=A;  
cout<<(*P)++<<endl;  
cout<<*P<<endl;
```

Soal 4.

```
int *P;  
int A[5]={2,1,7,0,5};  
P=A;  
cout<<*++P<<endl;  
cout<<*P<<endl;
```

Soal 5.

```
int *P;  
int A[5]={2,1,7,0,5};  
P=A;  
cout<<++*P<<endl;  
cout<<*P<<endl;
```

D. Referensi

Kirch-prinz U, Prinz P. *A Complete Guide to Programming in C++*. 1st ed. Sudbury, MA: Jones And Bartlett Publishers; 2002.

Arndt J. *Algorithms for Programmers.*; 2008.

Sjukani M. *Algoritma Dan Struktur Data 1 Dengan C, C++ Dan Java*. Jakarta, Indonesia: Mitra Wacana Media; 2014.

Edition E. *EIGHTH EDITION with an Introduction to C++ HOW TO PROGRAM*.

A.S. R. *Logika Algoritma Dan Pemrograman Dasar*. Bandung, Indonesia: Mandalajati; 2018.

Davis SR. *C++ for Dummies*. 7th Editio. New Jersey, USA: John Wiley & Sons, Inc; 2014.